

**UNIVERSIDADE ESTADUAL  
PAULISTA  
“JULIO DE MESQUITA FILHO”**

**Curso de Ciência da Computação**

**Compiladores**

Descrição da Linguagem Regular

Alunos: Rafaela Granado Castilho, Raí da Silva de Mattos

São José do Rio Preto- 2025

## Motivações

Como motivação para definirmos a linguagem, pensamos em algo semelhante ao Portugol. Atualmente, a programação exige um certo conhecimento do usuário para que ele consiga visualizar seus algoritmos funcionando. Dessa forma, decidimos criar uma linguagem que tenha elementos em nossa língua materna, o português.

Com isso, usuários que não têm conhecimento básico de inglês, mas desejam começar a programar, poderão ser atendidos de maneira mais acessível.

## Tipos de dados

Definimos inicialmente quatro tipos de dados básicos, baseados em linguagens de mais alto nível. Assim, o programador iniciante não precisará lidar com mecânicas de baixo nível, como vetores de caracteres.

O analisador léxico reconhecerá as seguintes palavras-chave:

- "inteiro" → números inteiros
- "real" → números reais
- "letra" → caracteres
- "palavra" → strings

## Palavras-chave

Seguindo a lógica de tradução dos termos em inglês, traduzimos também as estruturas de controle:

- if → se
- else → nao
- while → enquanto
- for → para
- return → retorne

Observa-se, entretanto, que os termos "nao" e "para" podem gerar ambiguidade ou confusão semântica. Em versões futuras do compilador, talvez seja necessário substituir esses identificadores.

## Literais

Os literais seguem o mesmo padrão de linguagens de alto nível.

- Inteiro: `[0-9]+` (ex.: 10, 2024)
- Real: `[0-9]+[.][0-9]+` (ex.: 3,14, 2.71)
- String: `"([^\"]|\\.)*"` (ex.: "teste", "linha nova")
- Caractere: `'[^\\n]'` (ex.: 'a', 'Z')

Números reais podem ser escritos tanto com vírgula quanto com ponto como separador decimal. Essa decisão foi tomada para facilitar a escrita: usuários iniciantes tendem a usar a vírgula (como na matemática escolar), enquanto programadores já acostumados com outras linguagens poderão utilizar o ponto.

## Atribuição

Para tornar os comandos de atribuição mais legíveis a iniciantes, definimos duas formas equivalentes:

- "=" ou "recebe" (ex.: `x = y`; `x recebe y`)

## Identificadores

A ideia foi diferenciar variáveis, funções e constantes por meio de padrões distintos de identificadores, de modo a facilitar a leitura.

No regex, utilizamos:

- `cA` → conjunto `{A-Z}`
- `cB` → conjunto `{a-z}`

Regras:

- Funções: `{cA}{cB}({cA}|{cB})*` (ex.: `Soma`, `Imprime`, `RecursoSintatico`)
- Constantes: `{cA}({cA}|[0-9])+` (ex.: `PI1`, `VALOR99`)
- Variáveis: `{cB}({cB}|[0-9])*` (ex.: `x`, `y2`, `total123`)

## Operadores

Adotamos os mesmos operadores do C, por serem padronizados e amplamente utilizados em diversas linguagens.

- Relacionais: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Lógicos: `&&`, `||`
- Aritméticos: `+`, `-`, `*`, `/`, `++`

## Delimitadores

A ideia é permitir uma indentação semelhante à do Python, reduzindo erros comuns de sintaxe (como esquecer `;` em C). Contudo, todos os delimitadores foram definidos para garantir flexibilidade nos próximos passos.

- Parênteses: `( )`
- Chaves: `{ }`
- Colchetes: `[ ]`
- Vírgula: `,`
- Ponto e vírgula: `;`
- Quebra de linha: `\n`

## Comentários

Foram adicionadas regras para comentários, que são reconhecidos como tokens, mas posteriormente ignorados pelo analisador léxico durante a análise sintática.

- Simples: `#.*` (ex.: `# expressão que calcula o delta`)
- Compostos: `/\^[^*]|\^[*/]|\^[*/]*/` (ex.: `/* esta função recebe a, b e c e retorna erro ou um real */`)